

A
PROJECT REPORT
ON
NATURAL LANGUAGE PROCESSING
CLASSIFICATION OF WEB TEXTS COMBINED WITH
DEEP LEARNING

AT
MANAC INFOTECH PVT LTD
SUBMITTED TO
FACULTY OF INFORMATICS
OSMANIA UNIVERSITY, HYDERABAD

In partial fulfillment of the requirement for the award of the Degree of
MASTER OF COMPUTER APPLICATIONS

BY
CHINDHAM ANJALI
H.T.NO. 118724862024

Under the guidance of

Dr. K. Tulasi
Designation



DEPARTMENT OF COMPUTER APPLICATIONS
SIDDHARTHA DEGREE COLLEGE FOR WOMEN
(Affiliated to Osmania University)
DILSUKHNAGAR, HYDERABAD — 500060.
2026



SIDDHARTHA DEGREE COLLEGE FOR WOMEN

(Affiliated to Osmania University)

Dilsukhnagar, Hyderabad- 500 060, Telangana.

Ph: 040-24064179, 24161721. E-mail id : gp.sdcw@gmail.com

College Code: 1187

CERTIFICATE

This is to certify that the Project Report entitled **NATURAL LANGUAGE PROCESSING CLASSIFICATION OF WEB TEXTS COMBINED WITH DEEP LEARNING** is the bonafide work done by **CHINDHAM ANJALI** bearing Hall Ticket No: **118724862024** submitted in the **DEPARTMENT OF COMPUTER APPLICATIONS, SIDDHARTHA DEGREE COLLEGE FOR WOMEN, DILSUKHNAGAR, HYDERABAD** under **Dr. K. Tulasi** guidance. She has completed Project work as per the rules prescribed for partial fulfillment of the award of MCA Programme.

INTERNAL GUIDE

HEAD OF THE DEPARTMENT

PRINCIPAL

Ref: M03/PROJ-26-27/S26023

PROJECT COMPLETION CERTIFICATE

This is to certify that the **CHINDHAM ANJALI (118724862024)** has successfully completed the Project on “**NATURAL LANGUAGE PROCESSING CLASSIFICATION OF WEB TEXTS COMBINED WITH DEEP LEARNING**” in Partial fulfillment of the **MCA Programme** for the duration of 90 days.

SOFTWARE: PYTHON

We wish her good luck in all future endeavors

Project Manager

An IIM Alumnus Enterprise

DECLARATION

I hereby declare that this Project Report entitled **NATURAL LANGUAGE PROCESSING CLASSIFICATION OF WEB TEXTS COMBINED WITH DEEP LEARNING** submitted by me to the **DEPARTMENT OF COMPUTER APPLICATIONS, SIDDHARTHA DEGREE COLLEGE FOR WOMEN, DILSUKHNAGAR, HYDERABAD** is a bonafide work undertaken by me and it is not submitted to any other University or Institution for the award of any degree/diploma/certificate or published any time before.

Date:

Place:

CHINDHAM ANJALI

ACKNOWLEDGEMENT

I would like to express my heartfelt thanks to my supervisor **Dr. K. Tulasi, Associate Professor** for the guidance throughout my Project Work.

I take privilege of expressing my thanks to **Ms. MEGHANA**, Project Coordinator in **MANAC INFOTECH PVT LTD**, Hyderabad giving me valuable advice and suggestions throughout the Project.

I express my gratitude to **Dr. G. NAGAI AH GARU**, Chairman of **SIDDHARTHA GROUP OF INSTITUTIONS** for providing me opportunity to pursue MCA Programme.

I would like to express my immense gratitude and sincere thanks to **Ms. KANCHANA TRIVENI**, Principal, **SIDDHARTHA DEGREE COLLEGE FOR WOMEN, DILSUKHNAGAR**, Hyderabad for giving me the opportunity to do the Project and for providing excellent academic environment in our college.

I take this opportunity to thank all those who have been of help to me in the completion of this Project. Finally, I thank my friends for their continuous support and help in the completion of my Project.

CHINDHAM ANJALI

S.No	CONTENTS	Page No
1	CHAPTER – 1 : PROJECT OVERVIEW	1-2
	1.1 Introduction	1
	1.2 Objective of the Project	2
	1.3 Scope of the Project	2
2	CHAPTER – 2 : RESEARCH AND REVIEW	3-4
	2.1 Existing Technologies	3
	2.2 Literature Analysis	3-4
	2.3 Comparative Study	4
3	CHAPTER – 3 : SYSTEM EVALUATION	5-6
	3.1 Current System	5
	3.2 Limitations of Current System	5
	3.3 Enhanced Proposed System	6
	3.4 Benefits of Proposed System	6
4	CHAPTER – 4 : TECHNICAL REQUIREMENTS	7
	4.1 Hardware Setup	7
	4.2 Software Setup	7
5	CHAPTER – 5 : PROJECT BLUEPRINT	8-22
	5.1 Architecture Design	8-9
	5.2 UML Models	9-10
	5.2.1 Use Case Diagram	10-11
	5.2.2 Class Diagram	11-12
	5.2.3 Sequence Diagram	12
	5.2.4 Collaboration Diagram	13
	5.2.5 Activity Diagram	14-15
	5.2.6 Component Diagram	15-16
	5.2.7 Deployment Diagram	16-17
	5.3 Database Structure	18-22

	5.3.1 ER Diagram	19
	5.3.2 Data Dictionary	20-22
6	CHAPTER – 6 : USER INTERFACE DESIGN	23
	6.1 Input Design	23
	6.2 Output Design	23
7	CHAPTER – 7 : SYSTEM IMPLEMENTATION	24-26
	7.1 Main Modules	24
	7.2 Module Explanation	24-26
8	CHAPTER – 8 : DEVELOPMENT ENVIRONMENT	27-53
	8.1 Python	27-32
	8.2 Django	33-38
	8.3 Database	39-42
	8.4 Domain	43-45
	8.5 Dataset	46-48
	8.6 Source code	48-53
9	CHAPTER – 9 : PERFORMANCE ANALYSIS	54-55
10	CHAPTER – 10 : RESULT VALIDATION	56-62
	10.1 Testing	56-59
	10.2 Screenshots	59-62
11	CHAPTER – 11 : FINAL SUMMARY	63-64
	11.1 Project Outcome	63
	11.2 Future Enhancements	64
12	CHAPTER – 12 : RESOURCE MATERIALS	65

NATURAL LANGUAGE PROCESSING CLASSIFICATION OF WEB TEXTS COMBINED WITH DEEP LEARNING

ABSTRACT

The project titled “Natural Language Processing Classification of Web Texts Combined with Deep Learning” focuses on developing an intelligent system for automatically classifying web-based textual content using Natural Language Processing (NLP) and Deep Learning techniques. The system analyzes large volumes of unstructured web texts such as blogs, online articles, reviews, news content, and social media posts, where manual classification becomes difficult and time-consuming. NLP techniques including tokenization, stop-word removal, stemming, lemmatization, and feature extraction are used to preprocess and understand textual data, while Deep Learning models such as Artificial Neural Networks (ANN), Recurrent Neural Networks (RNN), Long Short-Term Memory (LSTM), and Transformer-based models are used to identify patterns, semantic relationships, and contextual meaning within the text. The proposed system improves classification accuracy, efficiency, and automated decision-making by categorizing texts into predefined classes such as sentiment analysis, spam detection, topic classification, and fake news identification. The model is trained and tested using labeled datasets, and its performance is evaluated using metrics such as accuracy, precision, recall, and F1-score. Overall, the integration of NLP and Deep Learning provides an effective, scalable, and intelligent solution for web text classification in applications such as digital marketing, social media analysis, cybersecurity, e-learning, and online information management.

CHAPTER – 1

PROJECT OVERVIEW

1.1 Introduction

Natural Language Processing (NLP) and Deep Learning are emerging technologies that play a major role in analyzing, understanding, and processing textual information available on the internet. In today's digital world, a huge amount of web-based textual data such as blogs, news articles, social media posts, online reviews, and websites is generated every day. Managing and classifying this unstructured data manually is a difficult and time-consuming task. Therefore, intelligent automated systems are required to efficiently organize and classify web texts based on their content and meaning.

The proposed project focuses on the classification of web texts by combining Natural Language Processing techniques with Deep Learning models. NLP techniques are used to preprocess and analyze textual data through methods such as tokenization, stop-word removal, stemming, lemmatization, and feature extraction. These techniques help in improving text quality and preparing the data for efficient classification. After preprocessing, Deep Learning algorithms such as Artificial Neural Networks (ANN), Recurrent Neural Networks (RNN), Long Short-Term Memory (LSTM), and Transformer-based models are applied to identify hidden patterns, semantic relationships, and contextual meanings present in the text data.

The integration of NLP and Deep Learning improves the overall accuracy, efficiency, and reliability of the text classification process compared to traditional machine learning methods. The proposed system can automatically categorize web texts into different predefined classes such as sentiment analysis, spam detection, fake news identification, topic classification, and content recommendation. The model is trained and evaluated using large textual datasets and performance metrics such as accuracy, precision, recall, and F1-score are used to measure system effectiveness.

1.2 Objective of the Project

The main objective of the project is to develop an intelligent and automated system for efficiently classifying large amounts of web-based textual data using Natural Language Processing (NLP) and Deep Learning techniques. The project aims to preprocess and analyze unstructured web texts such as blogs, news articles, social media posts, and online reviews to identify meaningful patterns, semantic relationships, and contextual information. By using advanced Deep Learning models such as Artificial Neural Networks (ANN), Recurrent Neural Networks (RNN), Long Short-Term Memory (LSTM), and Transformer-based models, the system aims to improve text classification accuracy, reduce manual effort, and enhance automated decision-making capabilities. The project also focuses on providing faster, scalable, reliable, and efficient text categorization for applications such as sentiment analysis, spam detection, fake news identification, topic classification, cybersecurity, digital marketing, and online information management systems.

1.3 Scope of the Project

The scope of the project is broad and highly relevant in today's digital environment where large amounts of web-based textual data are continuously generated. The project can be applied in various domains such as social media analysis, digital marketing, cybersecurity, e-learning, online recommendation systems, customer feedback analysis, spam detection, sentiment analysis, fake news detection, and search engine optimization. By combining Natural Language Processing (NLP) with Deep Learning techniques, the system can efficiently process and classify unstructured web texts with improved accuracy and contextual understanding. The project also supports automated information management, reduces manual effort, and enhances real-time decision-making capabilities. In the future, the system can be further extended by integrating advanced Transformer-based models, multilingual text classification, real-time web data processing, voice-to-text analysis, and cloud-based deployment for handling large-scale applications more effectively and intelligently.

CHAPTER – 2

RESEARCH AND REVIEW

2.1 Existing Technologies

The existing technologies used in the project include Natural Language Processing (NLP), Deep Learning, Machine Learning, Python programming, and web-based text processing technologies. NLP techniques such as tokenization, stemming, lemmatization, stop-word removal, and feature extraction are used for preprocessing and understanding textual data. Deep Learning technologies including Artificial Neural Networks (ANN), Recurrent Neural Networks (RNN), Long Short-Term Memory (LSTM), and Transformer-based models are used for learning complex textual patterns and improving classification accuracy. Machine Learning libraries such as TensorFlow, Keras, Scikit-learn, and PyTorch help in model training, prediction, and evaluation. Python is used as the primary programming language because of its simplicity and rich ecosystem of NLP and Deep Learning libraries. Web technologies and datasets collected from online sources such as blogs, news articles, reviews, and social media posts are used for text analysis and classification. Additionally, data preprocessing, feature extraction, and performance evaluation technologies are used to improve the efficiency, scalability, and reliability of the overall system.

2.2 Literature Analysis

Literature analysis for this project shows that text classification has become an important research area in Natural Language Processing because of the rapid growth of web-based textual data. Earlier systems used traditional machine learning methods such as Naive Bayes, Support Vector Machine, Decision Tree, and Random Forest for classifying text based on manually extracted features like TF-IDF and Bag-of-Words. These methods performed well for small and structured datasets, but they had limitations in understanding context, semantic meaning, and complex language patterns present in web texts.

Recent studies have focused on Deep Learning models such as Artificial Neural Networks, Convolutional Neural Networks, Recurrent Neural Networks, LSTM, GRU, and Transformer-based models for improving text classification accuracy. These models automatically learn important features from text and can handle large volumes of unstructured web data more

effectively. Literature also shows that combining NLP preprocessing techniques with Deep Learning improves the quality of input data and increases classification performance. Overall, existing research proves that NLP combined with Deep Learning provides a more accurate, scalable, and intelligent approach for web text classification compared to traditional methods.

2.3 Comparative Study

The comparative study of the project analyzes the performance differences between traditional machine learning methods and modern Deep Learning techniques used for web text classification. Traditional algorithms such as Naive Bayes, Support Vector Machine (SVM), Decision Tree, and Random Forest mainly depend on manual feature extraction methods like Bag-of-Words and TF-IDF, which provide moderate accuracy but have limitations in understanding contextual meaning and semantic relationships in large unstructured web texts. In contrast, Deep Learning models such as Artificial Neural Networks (ANN), Recurrent Neural Networks (RNN), Long Short-Term Memory (LSTM), and Transformer-based models automatically learn important features from textual data and provide better contextual understanding, semantic analysis, scalability, and higher classification accuracy. The study also shows that combining Natural Language Processing techniques such as tokenization, stemming, lemmatization, stop-word removal, and feature extraction with Deep Learning significantly improves the efficiency, reliability, precision, recall, and overall performance of web text classification systems compared to traditional approaches.

CHAPTER – 3

SYSTEM EVALUATION

3.1 Current System

The existing approaches for web text classification use classic word vector representation methods, like Word2Vec and GloVe. Such methods capture semantic meanings to a certain degree, but still depend on static embedding that fail to accurately and efficiently show the contextual meaning of words. For classification tasks, we find that common deep learning models, such as, TextCNN, RNNs, and LSTMs have been used. TextCNNs usually focus on extracting local features of the text, while RNNs and LSTMs are designed to capture sequential dependencies in text. Although there is reasonable accuracy achieved with those methods, the inability to capture bidirectional and contextual meaning, as well as long-range dependencies, remain an obstacle while dealing with complex web text.

3.2 LIMITATIONS OF CURRENT SYSTEM:

- Word embeddings like Word2Vec and GloVe yield "static" vectors, because they fail to capture context-specific meanings.
- CNNs and RNNs alone exhibit a limited sense of local and global context information capture.
- It is difficult to extract deeper semantic relationships for words in long text.
- Surprisingly low classification accuracy and F1 scores compared to the current transformer-based methods.
- The reliance on manual feature engineering and low robustness in different text domains.

3.3 ENHANCED PROPOSED SYSTEM

The presented scheme developed the BERT-BGCA model, which uses Bidirectional Encoder Representations from Transformers (BERT) to integrate BIGRU, CNN, and an attention mechanism. BERT captures the deep contextual semantic relationships through dynamic context-sensitive word embedding mapped to the text for generating BERT embedding. BIGRU is used to model the sequencing dependencies in both backwards and forwards direction while the CNN is utilized to efficiently extract relevant local features. The attention mechanism improves interpretability, allowing the model to class the relevant portion of the text, increasing classification accuracies. Thus, the BERT-BGCA model is developed for classifying large-scale web text and achieves higher overall accuracies based on model performance.

3.4 BENEFITS OF PROPOSED SYSTEM

- Dynamic word embedding generate by BERT represent the contextualized meanings of words better than static embedding.
- Integrating BIGRU allows for the identification of bidirectional contextual dependencies present in the text sequence.
- The CNN layer adds additional focus on local feature extraction while capitalizing on the global features which BGRU learns.
- By employing the attention mechanism, we can improve the interpretability of the model and allow the model to focus on the important areas of the input.
- Attains a greater classification accuracy (95.21%) and F1 score (94.36%) which are better than Text CNN and remain superior than the other baselines.
- The framework provides a consistent and scalable method that can be applied to varying types of web texts.
- Since all feature extraction is automated there is decreased reliance on manual feature engineering.

CHAPTER – 4

TECHNICAL REQUIREMENTS

4.1 Hardware setup

Hardware Component	Requirement
Processor	Intel Core i3 or Above
RAM	4 GB Minimum
Storage	128 GB HDD/SSD
Monitor	Standard Display Monitor
Internet	Broadband / Mobile Hotspot

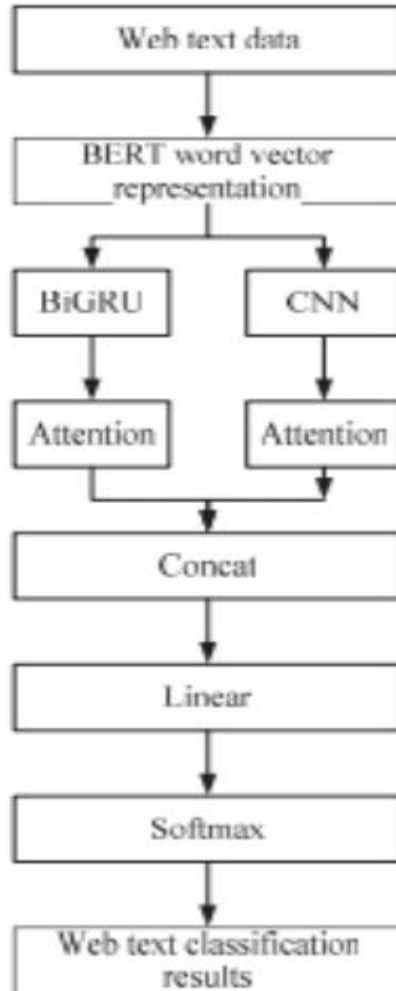
4.2 Software setup

Software Component	Requirement
Operating System	Windows 11
Programming Language	Python 3.11.9
Framework	Django
Frontend	HTML, CSS, JavaScript, Bootstrap
Database	MySQL 8.0
IDE	Visual Studio Code
Browser	Google Chrome
Design Software	Lucidchart

CHAPTER – 5

PROJECT BLUEPRINT

5.1 SYSTEM ARCHITECTURE:



The architecture shown in the diagram represents a Deep Learning-based web text classification system that combines BERT, BiGRU, CNN, and Attention mechanisms to improve text understanding and classification accuracy. The process starts with Web Text Data, where textual information such as online articles, blogs, reviews, or social media posts is collected as input data for classification.

The collected text data is then converted into meaningful numerical representations using BERT Word Vector Representation. BERT (Bidirectional Encoder Representations from Transformers) helps the system understand the contextual meaning of words by analyzing both

left and right contexts in a sentence. This improves semantic understanding and generates high-quality feature representations from the text.

After feature extraction using BERT, the architecture splits into two parallel processing paths: BIGRU and CNN. The BIGRU (Bidirectional Gated Recurrent Unit) captures sequential and contextual relationships between words by processing the text in both forward and backward directions. This helps the system understand long-term dependencies and sentence meaning. At the same time, the CNN (Convolutional Neural Network) extracts important local features and patterns such as key phrases, keywords, and textual structures from the web text data.

Next, an Attention Mechanism is applied to both the BIGRU and CNN outputs. The attention layer helps the model focus on the most important words and features within the text while ignoring irrelevant information. This improves feature selection and enhances classification performance.

The outputs from both attention layers are then combined in the Concat (Concatenation) Layer, where features learned from BIGRU and CNN are merged into a single feature representation. This combined information contains both contextual and local textual features, making the system more effective and accurate.

The merged features are passed to the Linear Layer, which performs mathematical transformations and prepares the data for final classification. After that, the Softmax Layer calculates the probability values for each predefined class and determines the most suitable category for the input text.

Finally, the system produces Web Text Classification Results, where the input web text is automatically classified into categories such as spam detection, sentiment analysis, fake news detection, topic classification, or content recommendation. Overall, this architecture improves contextual understanding, feature extraction, classification accuracy, and automated decision-making in web text analysis systems.

5.2 UML Models

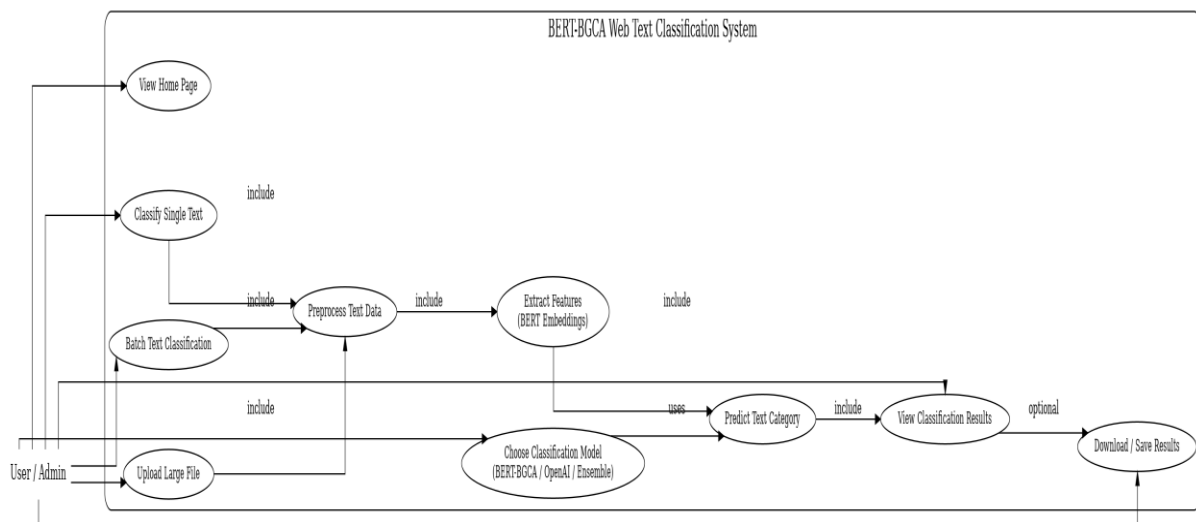
In the project the UML model is used to represent the overall design, workflow, and functionality of the system in a structured and organized manner. UML helps in understanding how different modules of the project interact with each other during the web text classification

process. It provides a clear visualization of system behavior, data flow, user interaction, and module communication, making the development process easier and more efficient.

The UML model mainly focuses on representing important system processes such as web text data collection, text preprocessing, feature extraction, Deep Learning-based classification, and result generation. It helps developers understand how the input text moves through different stages including tokenization, word vector generation, feature learning, attention mechanisms, and final classification. The model also explains the relationship between users, datasets, NLP modules, Deep Learning models, databases, and output systems.

Using UML modeling in this project improves system planning, software design, debugging, implementation, and documentation. It provides a better understanding of both the structural and behavioral aspects of the project and helps ensure smooth communication between developers, researchers, and users. Overall, the UML model plays an important role in building an efficient, scalable, and intelligent web text classification system using NLP and Deep Learning technologies.

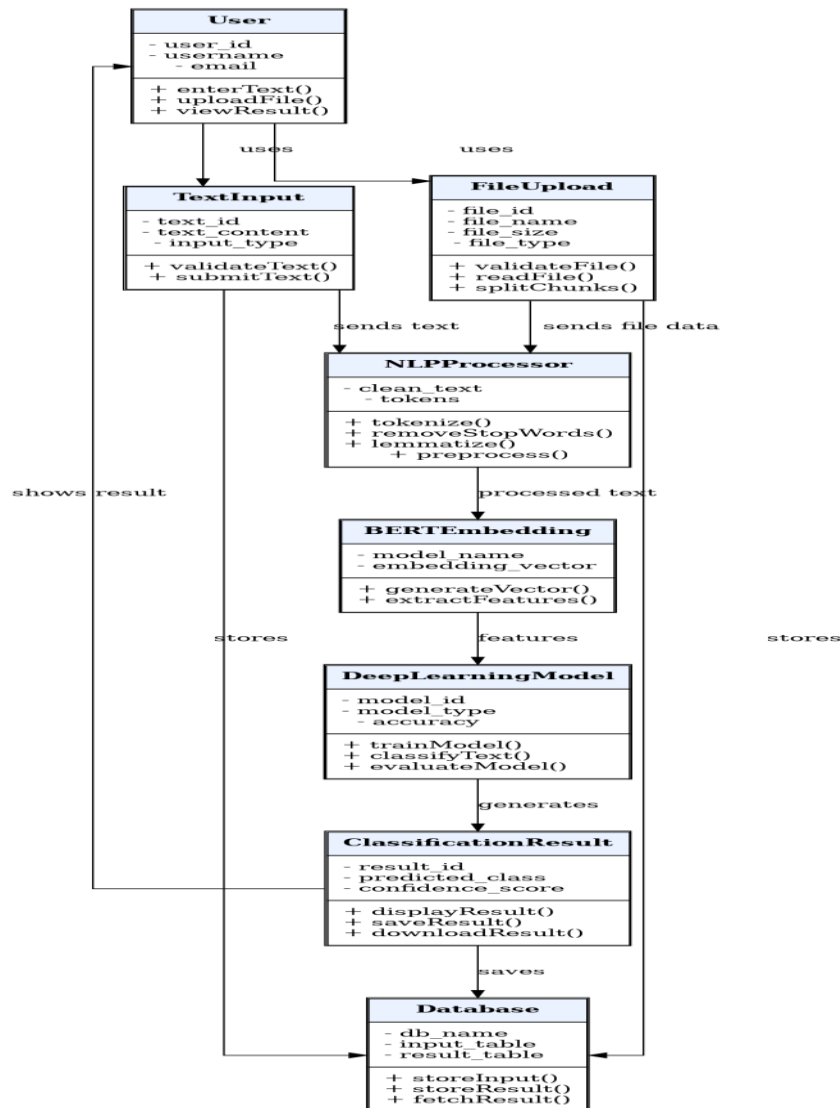
5.2.1 Use Case Diagram



The Use Case Diagram represents the interaction between the user and the BERT-BGCA Web Text Classification System. It explains how the user performs different operations such as viewing the home page, classifying single text, batch text classification, and uploading large files for processing. The system preprocesses the input text data, extracts important features using BERT embedding, and predicts the appropriate text category using Deep Learning

models. The user can also choose different classification models such as BERT-BGCA, OpenAI, or Ensemble methods for better accuracy. Finally, the system displays the classification results and allows users to save or download the output. Overall, the use case diagram helps in understanding the functionality, workflow, and user interaction within the web text classification system.

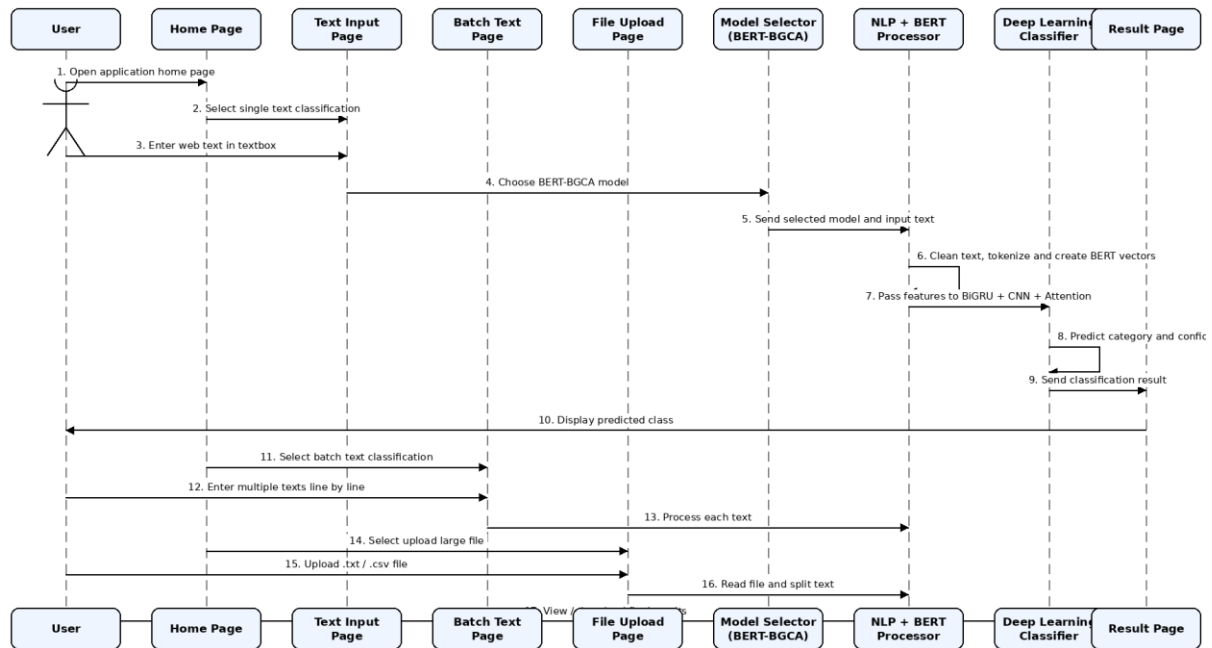
5.2.2 Class Diagram



The Class Diagram represents the structural design of the BERT-BGCA Web Text Classification System and shows the relationship between different classes used in the project. It includes important classes such as User, Text Input, File Upload, NLP Processor, BERT Embedding, Deep Learning Model, Classification Result, and Database. The User class interacts with the system by entering text and uploading files for classification. The NLP

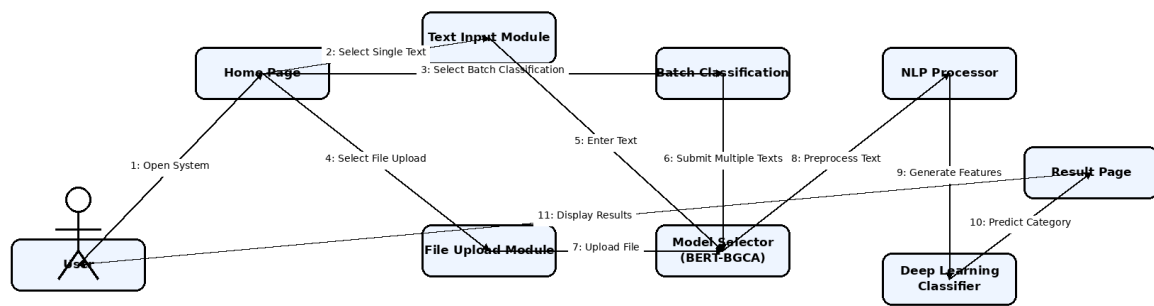
Processor class performs preprocessing operations such as tokenization and stop-word removal, while the BERT Embedding class extracts contextual features from the text. The Deep Learning Model class classifies the web text using Deep Learning techniques and predicts the appropriate category. Finally, the Classification Result and Database classes store, display, and manage the classification outputs efficiently.

5.2.3 Sequence Diagram



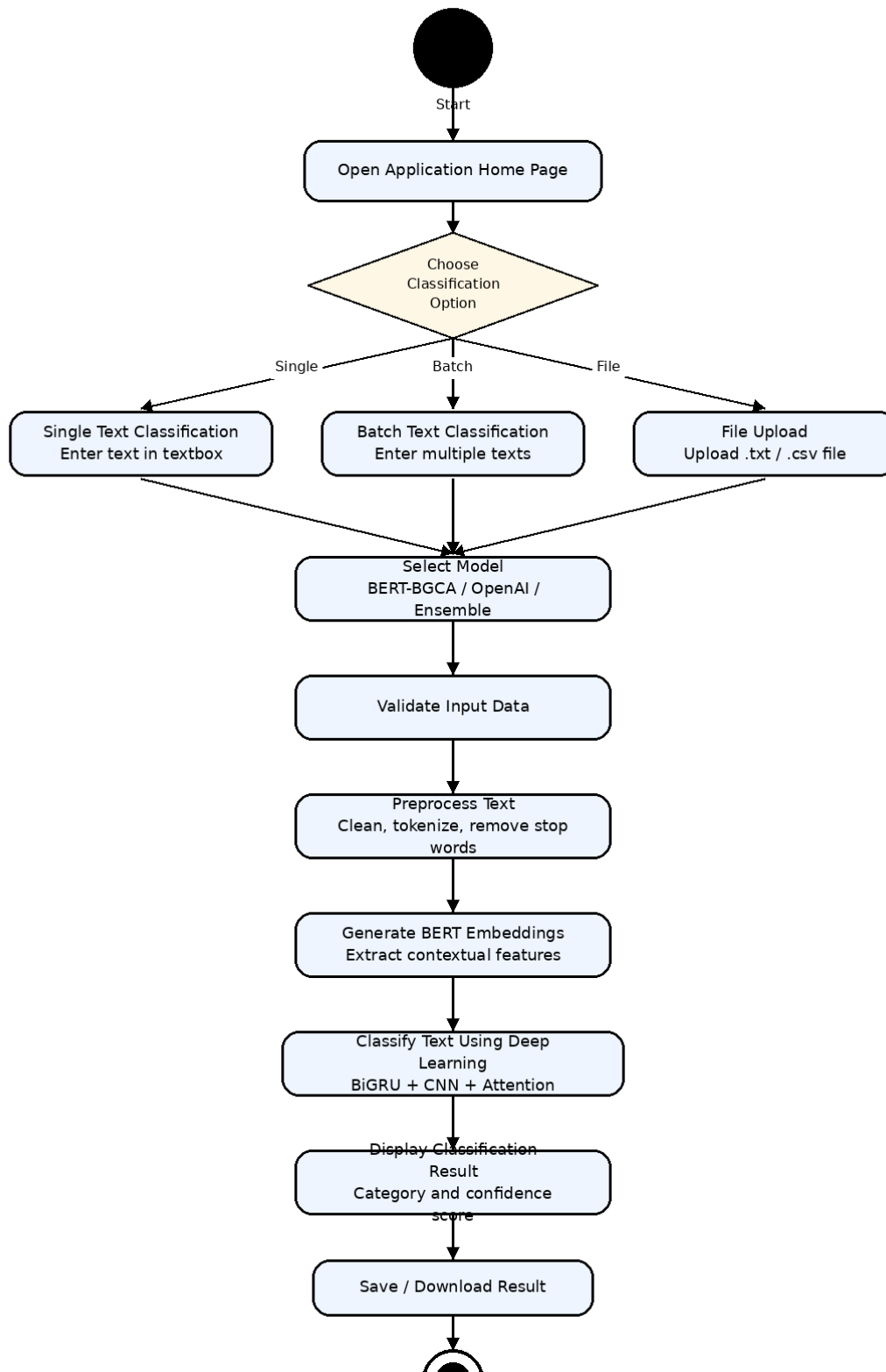
The Sequence Diagram represents the step-by-step interaction between the user and the NLP Web Text Classification System during the classification process. It starts when the user opens the home page and selects options such as single text classification, batch text classification, or file upload. The system then accepts the input text or uploaded file and sends it to the NLP and BERT processing module for preprocessing, tokenization, and feature extraction. After generating BERT embedding the extracted features are passed to the Deep Learning classifier, which uses BIGRU, CNN, and Attention mechanisms to predict the appropriate text category. The predicted result and confidence score are then sent to the result page and displayed to the user. Finally, the user can view, save, or download the classification results generated by the system.

5.2.4 Collaboration Diagram



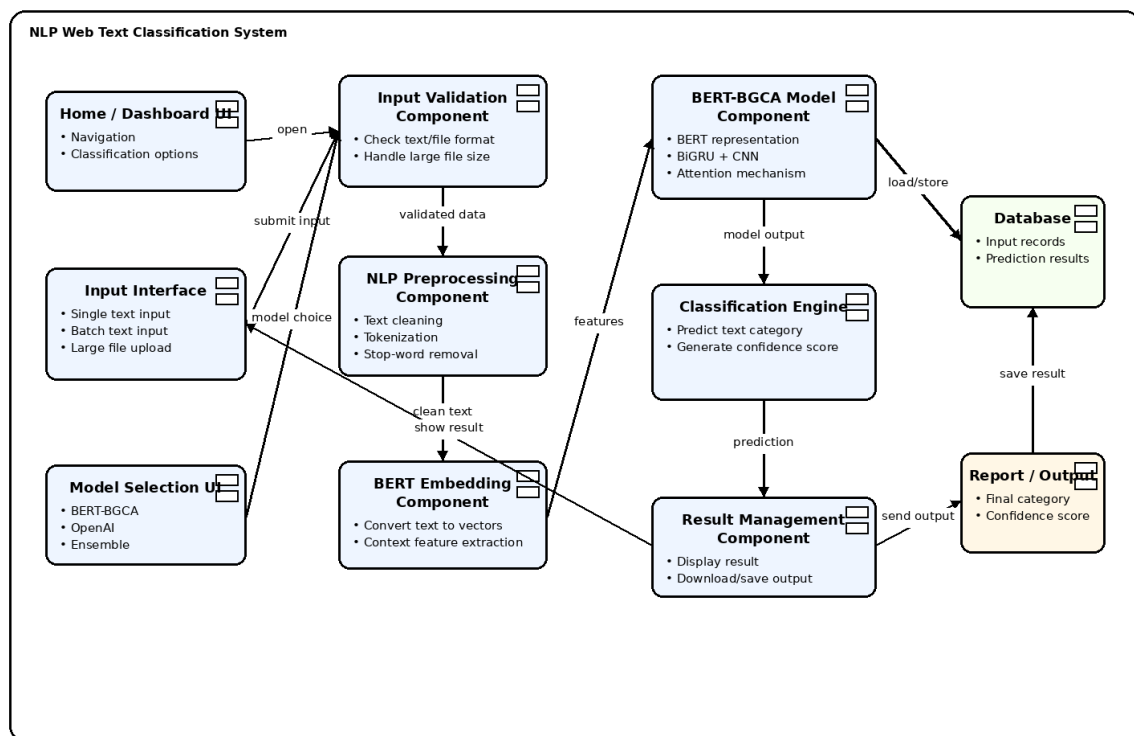
The Collaboration Diagram represents the communication and interaction between different modules of the NLP Web Text Classification System during the classification process. It shows how the user interacts with the home page to perform operations such as single text classification, batch text classification, and file upload. The entered text or uploaded file is then sent to the BERT-BGCA model selection module, where the appropriate classification model is chosen for processing. The NLP Processor preprocesses the text data and extracts important features using BERT embedding before sending them to the Deep Learning classifier. The Deep Learning module predicts the appropriate text category using techniques such as BiGRU, CNN, and Attention mechanisms. Finally, the classification results are displayed on the result page and provided to the user for viewing, saving, or downloading.

5.2.5 Activity Diagram



The Activity Diagram represents the workflow of the NLP Web Text Classification System from start to end during the classification process. It begins when the user opens the application and selects a classification option such as single text classification, batch text classification, or file upload. The user then enters the text data or uploads a file, and the system validates the input before processing it. After validation, the NLP module preprocesses the text by performing cleaning, tokenization, and stop-word removal, followed by BERT embedding generation for feature extraction. The extracted features are passed to the Deep Learning classifier, which uses BIGRU, CNN, and Attention mechanisms to predict the appropriate text category. Finally, the system displays the classification result with confidence score and allows the user to save or download the generated output.

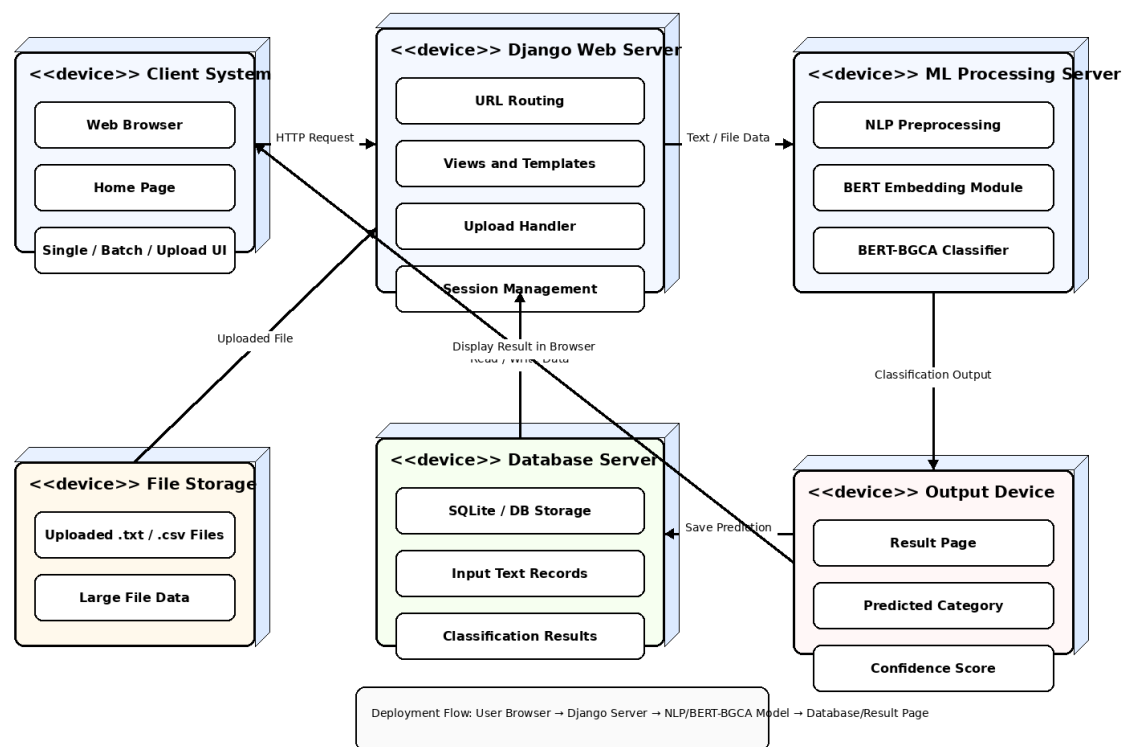
5.2.6 Component Diagram



The Component Diagram represents the overall software architecture of the NLP Web Text Classification System and shows how different system components are connected and interact with each other. It includes components such as the Home Page, Input Interface, Model Selection Module, NLP Preprocessing Component, BERT Embedding Component, BERT-BGCA Model, Classification Engine, Database, and Result Management Module. The user

interacts with the system through the input interface by entering text, batch data, or uploading files for classification. The NLP preprocessing component cleans and processes the text data before generating contextual BERT embeddings for feature extraction. These features are then passed to the BERT-BGCA classification engine, which uses Deep Learning techniques such as BiGRU, CNN, and Attention mechanisms to predict the appropriate text category. Finally, the classification results are stored in the database and displayed to the user through the result management component for viewing, saving, or downloading.

5.2.7 Deployment Diagram



The Deployment Diagram represents the physical architecture and deployment structure of the NLP Web Text Classification System. It shows how different hardware and software components are connected to perform web text classification operations. The Client System allows users to access the application through a web browser and perform tasks such as single text classification, batch classification, and file upload. The Django Web Server handles user requests, manages sessions, processes uploaded files, and communicates with the NLP and BERT-BGCA processing server. The ML Processing Server performs NLP preprocessing, BERT embedding generation, and Deep Learning-based classification using BiGRU, CNN, and Attention mechanisms. The Database Server stores input text records and classification results. The Output Device displays the predicted category and confidence score to the user.

and Attention mechanisms. The Database Server stores input texts, uploaded files, and classification results, while the Output Device displays the predicted category and confidence score to the user through the result page.

5.3 Database Structure

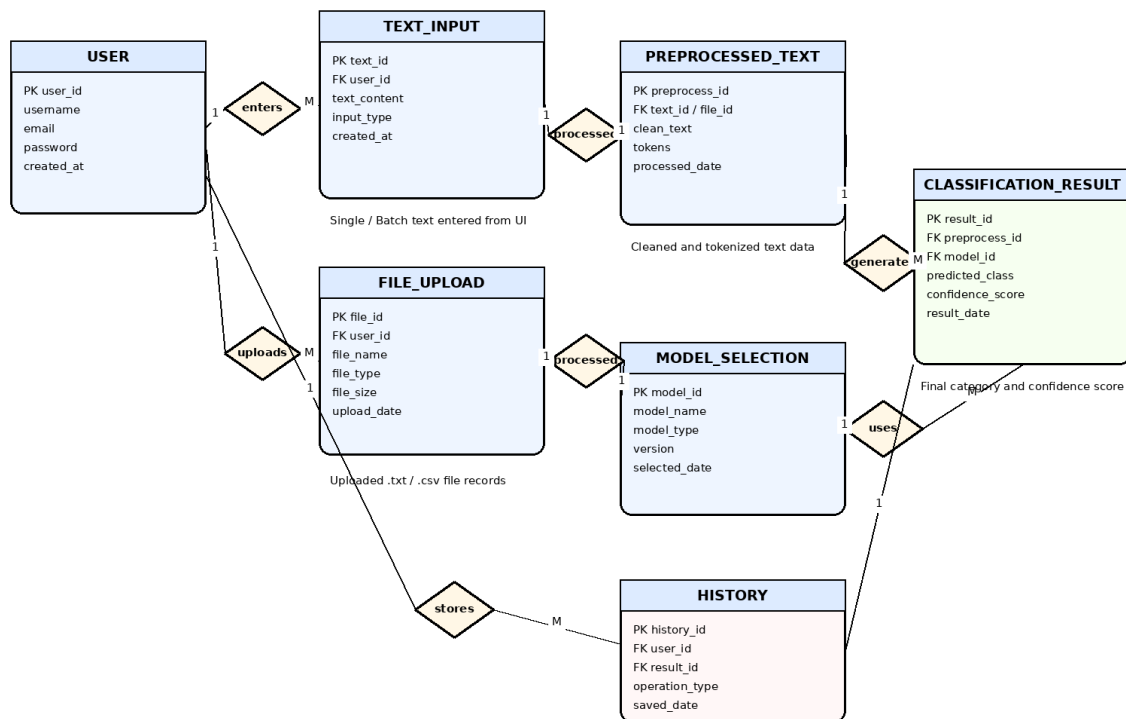
The database structure of the project is designed to efficiently store, manage, and retrieve web text data, uploaded files, preprocessing information, classification results, and user-related details. The database acts as the central storage system that maintains all the input data and prediction outputs generated during the text classification process. It helps in organizing the textual data systematically and supports efficient communication between different modules of the system such as the user interface, NLP processor, Deep Learning model, and result management system.

The database mainly contains tables such as User Table, Text Input Table, File Upload Table, Preprocessed Data Table, Classification Result Table, and Model Information Table. The User Table stores user details such as user ID, username, email, and login information. The Text Input Table stores single and batch web text data entered by the user for classification. The File Upload Table manages uploaded files such as .txt and .csv files along with file names, sizes, and upload timestamps.

The Preprocessed Data Table stores cleaned and tokenized textual information generated after NLP preprocessing operations such as stop-word removal, stemming, lemmatization, and tokenization. The Model Information Table stores details about the selected classification models such as BERT-BGCA, Open AI models, or Ensemble models used during prediction. The Classification Result Table stores the final predicted category, confidence score, classification status, and generated output results for future analysis and retrieval.

The database structure improves data organization, scalability, result tracking, and efficient system performance. It also supports storing historical classification records, monitoring model performance, and managing large-scale textual datasets effectively. Overall, the database structure plays an important role in ensuring reliable data management and smooth execution of the NLP and Deep Learning-based web text classification system.

5.3.1 ER Diagram



The ER Diagram represents the database structure of the NLP Web Text Classification System and shows the relationship between different entities in the project. It includes entities such as **USER**, **TEXT_INPUT**, **FILE_UPLOAD**, **PREPROCESSED_TEXT**, **MODEL_SELECTION**, and **CLASSIFICATION_RESULT**. The **USER** entity stores user details, while **TEXT_INPUT** and **FILE_UPLOAD** manage textual and uploaded file data. The **PREPROCESSED_TEXT** entity stores cleaned and tokenized text after NLP preprocessing. The **MODEL_SELECTION** entity stores information about the selected classification model such as BERT-BGCA. Finally, the **CLASSIFICATION_RESULT** entity stores the predicted category and confidence score generated by the Deep Learning classification system.

5.3.2 Data Dictionary

USERS

Field Name	Data Type	Description
user_id	Integer	Unique identifier for user session
input_type	Varchar	Type of input (single, batch, file upload)
selected_model	Varchar	Selected classification model
created_at	DateTime	Date and time of operation

INPUT_TEXTS

Field Name	Data Type	Description
text_id	Integer	Unique identifier for input text
user_id	Integer	Identifier for user/session
text_content	Text	Web text entered by the user
input_type	Varchar	Type of input (single or batch)
created_at	DateTime	Date and time of text submission

IDENTITIES

Field Name	Data Type	Description
identity_id	Integer	Unique identifier for classification identity
category_name	Varchar	Name of predicted text category
confidence_score	Float	Prediction confidence value
model_used	Varchar	Classification model used
classified_date	DateTime	Date and time of classification

T5_RESULTS

Field Name	Data Type	Description
result_id	Integer	Unique identifier for T5 result
text_id	Integer	Identifier for input text
extracted_text	Text	Text generated or extracted by T5 model
predicted_category	Varchar	Classified text category
confidence_score	Float	Prediction confidence value
processed_date	DateTime	Date and time of processing

KEYBERT_RESULTS

Field Name	Data Type	Description
keyword_id	Integer	Unique identifier for keyword result
text_id	Integer	Identifier for input text
extracted_keyword	Varchar	Keyword extracted using KeyBERT
relevance_score	Float	Importance score of keyword
extracted_date	DateTime	Date and time of keyword extraction

ANALYSIS_RECORDS

Field Name	Data Type	Description
analysis_id	Integer	Unique identifier for analysis record
text_id	Integer	Identifier for input text
model_used	Varchar	Model used for analysis
analysis_result	Text	Generated classification or analysis result
accuracy_score	Float	Accuracy or confidence score
analysis_date	DateTime	Date and time of analysis

LOGS

Field Name	Data Type	Description
log_id	Integer	Unique identifier for log record
user_id	Integer	Identifier for user/session
activity_type	Varchar	Type of activity performed
activity_status	Varchar	Status of operation (success/failure)
log_message	Text	Detailed system log message
created_at	DateTime	Date and time of log creation

PROCESSING_HISTORY

Field Name	Data Type	Description
history_id	Integer	Unique identifier for processing history
text_id	Integer	Identifier for input text
processing_step	Varchar	NLP or classification step performed
model_used	Varchar	Model used during processing
processing_status	Varchar	Status of processing operation
processed_date	DateTime	Date and time of processing

TEXT_SOURCES

Field Name	Data Type	Description
source_id	Integer	Unique identifier for text source
source_name	Varchar	Name of the text source or website
source_type	Varchar	Type of source (blog, news, social media, etc.)
source_url	Varchar	URL or link of the source
collected_date	DateTime	Date and time of data collection

SYSTEM_SETTINGS

Field Name	Data Type	Description
setting_id	Integer	Unique identifier for system setting
setting_name	Varchar	Name of the system setting
setting_value	Varchar	Value assigned to the setting
setting_description	Text	Description of the setting
updated_date	DateTime	Date and time of last update

CHAPTER – 6

USER INTERFACE DESIGN

6.1 Input Design

The input design focuses on collecting, validating, and processing web-based textual data efficiently for classification. The system accepts different types of input such as single text input, batch text input, and large file uploads in formats like .txt and .csv. Users can enter web texts manually through the text area or upload files containing multiple textual records for analysis. The input module also allows users to select classification models such as BERT-BGCA, Open AI, or Ensemble models before processing the data. During input processing, the system performs validation checks to ensure proper text format, remove invalid characters, and verify file size limitations. NLP preprocessing techniques such as tokenization, stop-word removal, stemming, and lemmatization are applied to clean and prepare the textual data before sending it to the Deep Learning classification module. The input design ensures accuracy, flexibility, scalability, and smooth handling of large volumes of unstructured web text data

6.2 Output Design

The output design of the project focuses on presenting classification results in a clear, accurate, and user-friendly format. After processing the input text using NLP and Deep Learning techniques, the system generates classification outputs such as predicted text category, confidence score, extracted keywords, sentiment result, spam detection result, or topic classification result depending on the selected operation. The results are displayed through the web interface in an organized manner so that users can easily understand the prediction generated by the BERT-BGCA classification model. The system also supports displaying batch classification results for multiple text records and allows users to save or download the generated outputs for future analysis. Additional output details such as processing status, selected model information, and prediction accuracy may also be shown to improve result interpretation. Overall, the output design improves usability, readability, result management, and efficient decision-making within the web text classification system.

CHAPTER – 7

SYSTEM IMPLEMENTATION

7.1 Main modules

1. User Interface Module
2. Text Input Module
3. File upload Module
4. NLP processing Module
5. BERT embedding Module
6. Deep learning classification Module
7. Model Selection Module
8. Result management Module
9. Database Management Module
10. Report and download Module

7.2 Module Explanation

1. User Interface Module

The User Interface Module is responsible for providing interaction between the user and the system through web pages. It allows users to enter single text, batch text, upload files, and select classification models such as BERT-BGCA, OpenAI, or Ensemble models. This module is developed using HTML, CSS, Bootstrap, and Django templates to provide a user-friendly and responsive environment for web text classification.

2. Text Input Module

The Text Input Module is used to accept textual data from users for classification. It supports single text input, multiple text entries, and batch processing for analyzing large amounts of web text data. The module validates the input text and forwards it to the NLP preprocessing module for further processing.

3. File Upload Module

The File Upload Module manages uploading and handling of large text files such as .txt and .csv. It checks file size, file format, and reads textual data from uploaded files before sending

the content for preprocessing and classification. This module helps in performing large-scale batch classification efficiently.

4. NLP Preprocessing Module

The NLP Preprocessing Module prepares the textual data for classification using Natural Language Processing techniques. It performs operations such as text cleaning, tokenization, stop-word removal, stemming, and lemmatization to improve text quality and remove unnecessary information. This module enhances the accuracy and efficiency of the Deep Learning classification system.

5. BERT Embedding Module

The BERT Embedding Module generates contextual word embedding and semantic feature representations from the preprocessed text. It uses BERT technology to understand the contextual meaning of words and improve feature extraction from web texts. The generated embedding are passed to the Deep Learning classification module for prediction.

6. Deep Learning Classification Module

The Deep Learning Classification Module performs the main classification task using advanced Deep Learning techniques such as BIGRU, CNN, and Attention mechanisms. This module analyse extracted features and predicts the appropriate category for the given web text. It improves classification accuracy, contextual understanding, and automated decision-making capabilities.

7. Model Selection Module

The Model Selection Module allows users to select different classification models available in the system such as BERT-BGCA, OpenAI models, or Ensemble techniques. It helps users compare classification performance and choose suitable models for different text analysis tasks.

8. Result Management Module

The Result Management Module displays the final classification results generated by the system. It shows outputs such as predicted category, confidence score, extracted keywords, and classification status in a user-friendly format. The module also supports saving and downloading classification reports for future use.

9. Database Management Module

The Database Management Module stores user inputs, uploaded files, preprocessing records, model information, and classification results. It manages data organization, retrieval, and storage efficiently to support system scalability and performance. The module also maintains historical records for future analysis and monitoring.

10. Report and Download Module

The Report and Download Module generates classification reports and downloadable output files for users. It allows users to save classification results, prediction summaries, and analysis reports in formats such as .txt or .csv. This module improves result management and helps users maintain classification records efficiently.

CHAPTER – 8

DEVELOPMENT ENVIRONMENT

8.1 Python

8.1.1 Python Overview

Python is a versatile, high-level, interpreted programming language created by Guido van Rossum and first released in 1991. It is governed by a design emphasises code readability and syntactic simplicity, captured in the famous document known as the Zen of Python (PEP 20). These qualities allow developers to express complex ideas in fewer lines of code compared to languages like Java or C++.

Python uses dynamic typing and automatic memory management through garbage collection. It supports multiple programming paradigms—procedural, object-oriented, and functional—making it suitable for an extremely wide variety of applications.

The language is backed by an enormous ecosystem through the Python Package Index (PyPI), which hosts over 450,000 third-party packages as of 2024. Whether the task is scientific computing, web development, automation, artificial intelligence, or DevOps scripting, a mature library almost certainly exists.

Python's global interpreter lock (GIL) is an important implementation detail: only one thread executes Python bytecode at a time within a single process. For CPU-bound parallelism, the multiprocessing module or external tools such as Disk or Ray are used instead. For I/O-bound tasks, the Asencio framework provides high-performance concurrency without threads.

Python versions follow a PEP 569 release cadence: a new feature release every October, with five years of security support. The 3.x branch (started 2008) is the only actively maintained line; Python 2 reached end-of-life on 1 January 2020.

8.1.2 Python 3.11.9 — Why This Version

Python 3.11 delivers the largest single-release performance improvement in the language's history — CPython benchmarks show 10–60 % speed gains over 3.10 due to the Specializing

Adaptive Interpreter (PEP 659), which optimises frequently executed bytecode sequences at runtime.

The 3.11.9 micro-release is a security and bug-fix update, making it the safest variant for production deployment.

8.1.3 Installation on Windows

Download the official installer and follow the steps below. Ensure Administrator privileges before running the installer.

STEP 1

Navigate to <https://www.python.org/downloads/release/python-3119/> and download the Windows installer (64-bit recommended for modern hardware).

STEP 2

Run the installer. On the first screen, tick the checkbox labelled 'Add Python 3.11 to PATH' before clicking anything else. Missing this step requires manual PATH configuration.

STEP 3

Click 'Customize Installation' to review optional features (pip, tcl/tk, documentation). For development work, leave all defaults selected.

STEP 4

Accept the Advanced Options. Confirm the install location (e.g. C:\Python311) and click Install.

STEP 5

Open Command Prompt and verify installation with the commands below.

```
# Verify Python version
python --version
# Expected output: Python 3.11.9

# Verify pip package manager
python -m pip --version
# Expected output: pip 24.x from C:\Python311\... (python 3.11)

# Confirm Python executable path
where python
```

Official Download – Python 3.11.9

8.1.4 Python Language Fundamentals

The following examples illustrate core language constructs used throughout the project. Each block is annotated to explain the intent.

Variables and Data Types

```
# — Integer: whole number without decimal —————
age = 25
# — String: immutable sequence of Unicode characters —————
name = 'John Doe'
# — List: ordered, mutable collection —————
scores = [85, 90, 78, 92]
# — Dictionary: key-value mapping (hash map) —————
student = {'name': 'Alice', 'grade': 'A', 'age': 20}
# — Tuple: ordered, immutable collection —————
coordinates = (10.5, 20.3)
# — Boolean: logical True / False —————
is_active = True
# — Float: double-precision decimal —————
gpa = 8.75
# — Set: unordered collection of unique elements —————
unique_tags = {'python', 'django', 'mysql'}
```

Control Flow

```
# — Conditional branching —————
if age >= 18:
    print('Adult')          # Executes when condition is True
elif age >= 13:
    print('Teenager')
else:
    print('Child')
# — For loop: iterate over an iterable —————
for score in scores:
    print(f'Score: {score}') # f-string interpolation
# — While loop with counter —————
count = 0
while count < 5:
    print(f'Count: {count}')
    count += 1              # Increment to avoid infinite loop
```

Functions

```
def calculate_average(numbers: list) -> float:
    '''Calculate arithmetic mean of a list of numbers.
```

```

Args:
    numbers: List of numeric values.
Returns:
    Float average, or 0.0 for empty list.
'''
if not numbers:          # Guard clause for empty input
    return 0.0
return sum(numbers) / len(numbers)

# — Function with default argument —————
def greet(name: str, greeting: str = 'Hello') -> str:
    return f'{greeting}, {name}!'

# — Lambda: anonymous one-liner function —————
square = lambda x: x ** 2

```

Object-Oriented Programming

```

class Student:
    '''Represents a university student record.'''

    # Class variable shared by all instances
    institution = 'Tech University'

    def __init__(self, name: str, roll_no: str, gpa: float):
        # Instance variables unique to each object
        self.name = name
        self.roll_no = roll_no
        self.gpa = gpa

    def display_info(self) -> None:
        '''Print student details to stdout.'''
        print(f'Name: {self.name} | Roll: {self.roll_no} | GPA: {self.gpa}')

    @property
    def grade_letter(self) -> str:
        '''Derive letter grade from GPA (property, not method).'''
        if self.gpa >= 9.0: return 'A+'
        if self.gpa >= 8.0: return 'A'
        if self.gpa >= 7.0: return 'B'
        return 'C'

    def __repr__(self) -> str:

```

```

        return f'Student({self.name!r}, gpa={self.gpa})'

# Inheritance -----
class GraduateStudent(Student):
    def __init__(self, name, roll_no, gpa, thesis_title):
        super().__init__(name, roll_no, gpa)    # Call parent constructor
        self.thesis_title = thesis_title

s1 = Student('Raj Kumar', 'MT001', 8.5)
s1.display_info()
print(s1.grade_letter)    # Output: A

```

Exception Handling

```

try:
    number = int(input('Enter a number: '))    # May raise ValueError
    result = 10 / number                        # May raise ZeroDivisionError
    print(f'Result: {result:.2f}')
except ValueError:
    print('Error: Please enter a valid integer.')
except ZeroDivisionError:
    print('Error: Cannot divide by zero.')
except Exception as e:
    print(f'Unexpected error: {e}')            # Catch-all fallback
else:
    print('Computation successful.')            # Runs only if no exception
finally:
    print('Operation completed.')              # Always runs

```

List Comprehensions and Generators

```

# — List comprehension: concise list creation -----
squares = [x**2 for x in range(10)]
# Output: [0, 1, 4, 9, 16, 25, 36, 49, 64, 81]

# — Conditional comprehension -----
even_squares = [x**2 for x in range(20) if x % 2 == 0]

# — Dictionary comprehension -----
word_lengths = {word: len(word) for word in ['python', 'django', 'mysql']}

# — Generator expression: lazy evaluation (memory-efficient) -----
large_gen = (x**2 for x in range(10_000_000))    # Does not allocate 10M list
first_five = [next(large_gen) for _ in range(5)]

```

8.1.5 Virtual Environment Setup

Virtual environments provide isolated Python installations per project, preventing dependency version conflicts. Every project should use its own virtual environment.

```
# — Create a new virtual environment named 'venv' —————  
python -m venv venv  
  
# — Activate on Windows (Command Prompt) —————  
venv\Scripts\activate  
  
# — Activate on Windows (PowerShell) —————  
venv\Scripts\Activate.ps1  
  
# — Activate on macOS / Linux —————  
source venv/bin/activate  
  
# — Prompt changes to (venv) when active —————  
# (venv) C:\project>  
  
# — Install packages inside the environment —————  
pip install django mysqlclient numpy pandas  
  
# — Freeze current dependencies to a file —————  
pip freeze > requirements.txt  
  
# — Recreate environment from requirements on another machine —————  
pip install -r requirements.txt  
  
# — Deactivate when done —————  
deactivate
```

8.2 Django Framework

8.2.1 Django Overview

Django is a free and open-source, high-level Python web framework first released in 2005. It was originally developed at the Lawrence Journal-World newspaper to manage fast-moving news sites, and open-sourced by the Django Software Foundation. The framework follows the Model-View-Template (MVT) architectural pattern—a Django-specific variant of the classical Model-View-Controller (MVC) separation.

Django's guiding philosophy is captured in the phrase 'batteries included': the framework ships with an ORM, an automatic admin panel, a URL dispatcher, a template engine, a form library, an authentication system, middleware infrastructure, and dozens of other production-ready components. Developers rarely need external libraries for common tasks.

Key design principles that govern Django development are:

- DRY (Don't Repeat Yourself) — logic is defined once and reused across the project.
- Loose coupling — each layer (model, view, template) can be replaced independently.
- Explicit over implicit — behaviour is never hidden or magically inferred.
- Security by default — CSRF tokens, SQL-injection-safe ORM, XSS-escaping templates, and clickjacking protection are all active out of the box.

Django powers major sites including Instagram, Pinterest, Disqus, and NASA's public websites, demonstrating its suitability for high-traffic, production workloads.

8.2.2 Django MVT Architecture

Understanding the MVT request lifecycle is essential for building well-structured Django applications:

- Model: Defines the data schema as Python classes. Django's ORM translates model operations into SQL and executes them against the configured database. Changes to models are version-controlled through migrations.
- View: A Python callable (function or class) that receives an HTTP request object, executes business logic (querying models, calling services), and returns an HTTP response. Views are the 'controller' layer in MVC terms.
- Template: An HTML file augmented with Django Template Language (DTL) tags and filters. Templates receive a context dictionary from the view and render it to a string that forms the HTTP response body.
- URL Dispatcher: Maps URL patterns to view callable. Uses regular expressions or path converters. URL namespaces prevent naming collisions in large multi-app projects.

8.2.3 Installation and Project Setup

Install Django inside an activated virtual environment, then scaffold the project structure using `django-admin`.

STEP 1

Activate your virtual environment and install Django 4.2 LTS (Long-Term Support, supported until April 2026).

```
# Install Django LTS release
pip install django==4.2

# Confirm version
python -m django --version    # Should output: 4.2
```

STEP 2

Create the Django project and a first application.

```
# Scaffold a new Django project (creates outer myproject/ directory)
django-admin startproject myproject

# Enter the project directory
cd myproject

# Create a Django application (reusable component within the project)
python manage.py startapp myapp
```

STEP 3

Run the built-in development server to verify the installation.

```
# Launch development server on default port 8000
python manage.py runserver

# Launch on a custom host:port (useful when testing on a LAN)
python manage.py runserver 0.0.0.0:8080

# Starting development server at http://127.0.0.1:8000/
# Quit the server with CTRL-BREAK.
```

8.2.4 Project Structure

```
myproject/
├── manage.py          # CLI utility for admin tasks (runserver, migrate...)
```

```

├─ requirements.txt      # Pinned dependencies
├─ myproject/           # Inner package – project configuration
│   └─ __init__.py      # Marks directory as Python package
│   └─ settings.py      # All project settings (DB, apps, middleware...)
│   └─ urls.py          # Root URL configuration
│   └─ asgi.py          # ASGI entry point (async servers, WebSockets)
│   └─ wsgi.py          # WSGI entry point (traditional web servers)
├─ myapp/               # Application package
│   └─ migrations/      # Auto-generated database migration files
│       └─ __init__.py
│   └─ admin.py         # Admin panel registration
│   └─ apps.py          # App configuration class
│   └─ models.py        # Database schema definitions
│   └─ tests.py         # Unit tests
│   └─ views.py         # Request handlers
│   └─ urls.py          # App-level URL patterns
│   └─ forms.py         # Django Form classes
│   └─ serializers.py   # DRF serializers (if using REST framework)
│   └─ templates/
│       └─ myapp/       # Namespaced templates
│           └─ base.html

```

8.2.5 Django Models

Models are the single authoritative source of truth about data. Each model class maps to one database table. Django's ORM handles all SQL generation, making database code portable across MySQL, PostgreSQL, and SQLite.

```

from django.db import models
from django.core.validators import MinValueValidator, MaxValueValidator

class Department(models.Model):
    '''Academic department (e.g., Computer Science).'''
    name = models.CharField(max_length=100, unique=True)
    code = models.CharField(max_length=10, unique=True)    # e.g. 'CS'
    established = models.PositiveIntegerField()            # Year founded

    def __str__(self):
        return self.name

class Student(models.Model):
    '''University student linked to a department.'''
    name = models.CharField(max_length=100)
    email = models.EmailField(unique=True)
    roll_number = models.CharField(max_length=20, unique=True)

```

```

gpa          = models.FloatField(
    default=0.0,
    validators=[MinValueValidator(0.0), MaxValueValidator(10.0)]
)
department    = models.ForeignKey(
    Department,
    on_delete=models.SET_NULL,    # Keep student if dept is deleted
    null=True,
    related_name='students'      # Reverse accessor: dept.students.all()
)
enrollment_date = models.DateField(auto_now_add=True) # Set on creation
is_active      = models.BooleanField(default=True)

class Meta:
    ordering = ['name']          # Default queryset order
    verbose_name_plural = 'Students'
    indexes = [
        models.Index(fields=['roll_number']), # Speed up roll lookups
    ]

def __str__(self):
    return f'{self.name} ({self.roll_number})'

```

8.2.6 Views and URLs

```

# views.py - request handlers
from django.shortcuts import render, get_object_or_404
from django.http import JsonResponse
from django.views import View
from .models import Student

# — Function-based view —————
def student_list(request):
    '''Return paginated list of active students.'''
    students = Student.objects.filter(is_active=True).select_related('department')
    # select_related avoids N+1 SQL queries by JOINing department
    return render(request, 'myapp/student_list.html', {'students': students})

# — Class-based view —————
class StudentDetailView(View):
    def get(self, request, pk):
        student = get_object_or_404(Student, pk=pk) # Returns 404 if missing
        return render(request, 'myapp/student_detail.html', {'student': student})

# — JSON API endpoint —————

```

```

def student_api(request, pk):
    student = get_object_or_404(Student, pk=pk)
    data = {
        'id': student.pk,
        'name': student.name,
        'email': student.email,
        'gpa': student.gpa,
        'department': str(student.department),
    }
    return JsonResponse(data)

# urls.py - URL routing
from django.urls import path
from . import views
app_name = 'myapp'    # Namespace for {% url 'myapp:student_list' %}

urlpatterns = [
    path('students/', views.student_list, name='student_list'),
    path('students/<int:pk>/', views.StudentDetailView.as_view(),
name='student_detail'),
    path('api/students/<int:pk>/', views.student_api, name='student_api'),
]

```

8.2.7 Templates

```

{# templates/myapp/student_list.html #}
<!DOCTYPE html>
<html lang='en'>
<head>
    <meta charset='UTF-8'>
    <title>Student List - {{ request.site.name }}</title>
</head>
<body>
    <h1>Students</h1>
    {# Display flash messages (success / error) #}
    {% for message in messages %}
        <p class='message' {{ message.tags }}>{{ message }}</p>
    {% endfor %}

    <ul>
    {% for student in students %}
        <li>
            {# url tag resolves to /students/<pk>/ #}
            <a href='{% url "myapp:student_detail" student.pk %}'>
                {{ student.name }} &mdash; {{ student.department }}
            </a>

```

```
        <span>GPA: {{ student.gpa|floatformat:2 }}</span>
    </li>
    {% empty %}
        <li>No students found.</li>    {% Shown when queryset is empty %}
    {% endfor %}
</ul>
</body>
</html>
```

8.3 Database — MySQL 8.0

8.3.1 MySQL 8.0 Overview

MySQL is the world's most popular open-source relational database management system, owned and maintained by Oracle Corporation. It uses a client-server model: the `mysqld` server daemon manages data files on disk, while clients connect over TCP/IP (default port 3306) or a Unix domain socket.

MySQL 8.0 introduced major features that close the gap with PostgreSQL: window functions (`RANK`, `DENSE_RANK`, `LAG`, `LEAD`), Common Table Expressions (CTEs) including recursive CTEs, native JSON data type with `JSON_TABLE()`, invisible indexes for safe index testing, roles for simplified permission management, and the `utf8mb4` character set as the new default—enabling full Emoji and supplementary Unicode support.

8.3.2 Installation on Windows

STEP 1

Go to <https://dev.mysql.com/downloads/installer/> and download the MySQL Installer for Windows (`mysql-installer-community-8.0.x.msi`).

STEP 2

Run the installer as Administrator. Choose 'Developer Default' (installs MySQL Server, Workbench, Shell, and ODBC driver) or 'Server only' for a minimal footprint.

STEP 3

On the 'Type and Networking' screen, confirm the port is 3306, select 'Development Machine' config type, and enable 'Start MySQL Server at System Startup'.

STEP 4

Set a strong root password on the 'Accounts and Roles' screen. Optionally add a dedicated application user with limited privileges.

STEP 5

Complete the wizard. Then verify the installation from Command Prompt.

```
# Check MySQL server version
mysql --version
# Expected: mysql Ver 8.0.x Distrib 8.0.x, for Win64 (x86_64)

# Log in as root (will prompt for password)
mysql -u root -p
```

```
# Inside MySQL shell – check server status
mysql> SHOW VARIABLES LIKE 'version%';
mysql> STATUS;
mysql> EXIT;
```

8.3.3 Creating Databases and Tables

```
-- — Connect to MySQL server —————
mysql -u root -p

-- — Create application database with full Unicode support —————
CREATE DATABASE student_management
    CHARACTER SET utf8mb4
    COLLATE utf8mb4_unicode_ci;

-- — Switch context to the new database —————
USE student_management;

-- — Departments table —————
CREATE TABLE departments (
    id            INT            PRIMARY KEY AUTO_INCREMENT,
    name          VARCHAR(100) NOT NULL UNIQUE,
    code          VARCHAR(10)  NOT NULL UNIQUE,
    established   YEAR          NOT NULL
);

-- — Students table with foreign key to departments —————
CREATE TABLE students (
    id            INT            PRIMARY KEY AUTO_INCREMENT,
    name          VARCHAR(100)  NOT NULL,
    email         VARCHAR(100)  NOT NULL UNIQUE,
    roll_number   VARCHAR(20)   NOT NULL UNIQUE,
    gpa           DECIMAL(4,2)  DEFAULT 0.00
        CHECK (gpa BETWEEN 0.00 AND 10.00),
    department_id INT,
    enrollment_date DATE,
    is_active     TINYINT(1)    DEFAULT 1,
    created_at    TIMESTAMP     DEFAULT CURRENT_TIMESTAMP,
    updated_at    TIMESTAMP     DEFAULT CURRENT_TIMESTAMP
        ON UPDATE CURRENT_TIMESTAMP,
    FOREIGN KEY (department_id)
        REFERENCES departments(id)
        ON DELETE SET NULL
```

```
);

-- — Index for fast GPA range queries —————
CREATE INDEX idx_students_gpa ON students (gpa);

-- — Verify tables —————
SHOW TABLES;
DESCRIBE students;
```

8.3.4 Django-MySQL Integration

```
# Install the MySQL client library (C-extension, requires MySQL headers)
pip install mysqlclient==2.2.0

# Alternative pure-Python driver (slower but easier to install on Windows)
pip install PyMySQL

# settings.py — database configuration
import os
from dotenv import load_dotenv

load_dotenv()    # Read .env file into environment

DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.mysql',
        'NAME': os.getenv('DB_NAME', 'student_management'),
        'USER': os.getenv('DB_USER', 'root'),
        'PASSWORD': os.getenv('DB_PASSWORD', ''),
        'HOST': os.getenv('DB_HOST', 'localhost'),
        'PORT': os.getenv('DB_PORT', '3306'),
        'OPTIONS': {
            'charset': 'utf8mb4',
            'init_command': "SET sql_mode='STRICT_TRANS_TABLES'",
            'connect_timeout': 10,
        }
    }
}
```

8.3.5 Migrations

```
# — Generate migration files from model changes —————
python manage.py makemigrations

# — Preview SQL without applying —————
```

```
python manage.py sqlmigrate myapp 0001

# — Apply all pending migrations —————
python manage.py migrate

# — View migration status —————
python manage.py showmigrations

# — Rollback to a specific migration —————
python manage.py migrate myapp 0002    # Reverts migrations after 0002

# — Restore from compressed backup —————
gunzip < backup.sql.gz | mysql -u root -p student_management
```

8.4 Domain

Python's strength lies in its rich ecosystem of specialised libraries. This section covers the key libraries across four domains used in this project: Machine Learning, Deep Learning, Each subsection includes theory, a library comparison table, and annotated code examples.

8.4.1 Machine learning ecosystem

Machine Learning (ML) is a branch of artificial intelligence that enables systems to learn patterns from data and improve performance without being explicitly programmed. Instead of writing rules, developers provide labelled examples (supervised learning), unlabelled data (unsupervised learning), or reward signals (reinforcement learning), and the algorithm discovers the underlying structure.

The ML pipeline typically follows these stages: data collection → preprocessing → feature engineering → model selection → training → evaluation → hyperparameter tuning → deployment → monitoring. Python dominates this workflow because its numeric and data libraries compose seamlessly.

Library	Primary Purpose	Installation
scikit-learn 1.3	Classical ML: classification, regression, clustering, preprocessing	pip install scikit-learn
XGBoost	Gradient boosted trees — top performer on tabular data	pip install xgboost
LightGBM	Fast gradient boosting with low memory usage	pip install lightgbm
pandas 2.0	DataFrame-based data manipulation and analysis	pip install pandas
NumPy 1.25	N-dimensional arrays and numerical computing	pip install numpy
Matplotlib 3.7	Static 2-D plots and publication-quality figures	pip install matplotlib

Library	Primary Purpose	Installation
Seaborn	Statistical data visualisation built on Matplotlib	pip install seaborn
Plotly	Interactive web-based charts and dashboards	pip install plotly
SHAP	Model explainability — SHapley Additive exPlanations	pip install shap
imbalanced-learn	Techniques for class-imbalanced datasets (SMOTE...)	pip install imbalanced-learn

8.4.2 Deep Learning Ecosystem

Deep Learning (DL) is a subfield of ML that uses artificial neural networks with many hidden layers (hence 'deep') to model highly non-linear relationships. The key advantage over classical ML is automatic feature extraction: a DL model learns to identify edges, textures, and shapes from raw pixels without hand-engineered features.

DL has achieved state-of-the-art performance in image classification (ResNet, EfficientNet), object detection (YOLO, Faster R-CNN), natural language processing (Transformer, BERT, GPT), speech recognition, and generative modelling (GANs, Diffusion Models). These advances depend on three enablers: large labelled datasets, GPU/TPU acceleration, and scalable frameworks.

Library	Primary Purpose	Installation
TensorFlow 2.13	End-to-end ML platform with Keras high-level API	pip install tensorflow
PyTorch 2.0	Dynamic computation graph, research-friendly	pip install torch torchvision
Keras 2.13	High-level API (now part of TensorFlow)	pip install keras
Hugging Face Transformers	Pre-trained NLP and vision transformers	pip install transformers

Library	Primary Purpose	Installation
OpenCV 4.8	Computer vision, image processing, video analysis	pip install opencv-python
NLTK 3.8	Classical NLP: tokenisation, POS tagging, parsing	pip install nltk
spaCy 3.6	Industrial NLP: NER, dependency parsing	pip install spacy
Pillow	Image loading, manipulation, and format conversion	pip install Pillow
torchmetrics	Standardised PyTorch metrics for training loops	pip install torchmetrics

8.5 Dataset

Dataset Information

The dataset used in the project mainly consists of web-based textual data collected from online sources such as news articles, blogs, social media posts, product reviews, discussion forums, and websites. The dataset contains unstructured text data that is used for training, testing, and evaluating the Deep Learning classification model. Different categories of textual data such as positive and negative sentiments, spam and non-spam messages, fake and real news content, technology articles, sports news, educational content, and cybersecurity-related texts are included in the dataset to improve classification accuracy and model performance.

The dataset undergoes several preprocessing operations such as text cleaning, tokenization, stop-word removal, stemming, lemmatization, and feature extraction before being used for classification. BERT embedding techniques are applied to generate contextual word representations from the dataset, which are then processed using Deep Learning models such as BIGRU, CNN, and Attention mechanisms. The dataset is divided into training and testing sets to evaluate system performance using metrics such as accuracy, precision, recall, and F1-score. Overall, the dataset plays a major role in improving the learning capability, contextual understanding, and classification efficiency of the NLP-based web text classification system.

Dataset Features Table

Feature Name	Description
text_id	Unique identifier for each text record
text_content	Original web text collected from online sources
cleaned_text	Preprocessed text after cleaning and filtering
tokens	Tokenized words generated during NLP preprocessing
source_type	Type of source such as blog, news, review, or social media
category_label	Assigned classification category for the text
sentiment	Sentiment type such as positive, negative, or neutral
keyword_features	Important keywords extracted from the text
bert_embeddings	Contextual feature vectors generated using BERT
confidence_score	Prediction confidence generated by the model

input_type	Type of input such as single text, batch text, or file upload
processed_date	Date and time when the text was processed

Dataset Types Table

Dataset Type	Description
News Dataset	Contains news articles collected from online news websites for topic classification and fake news detection
Social Media Dataset	Includes social media posts, comments, and tweets used for sentiment analysis and text classification
Review Dataset	Contains customer reviews and feedback for positive, negative, and neutral sentiment prediction
Spam Dataset	Includes spam and non-spam textual messages used for spam detection classification
Blog Dataset	Contains blog articles and web content used for topic-based text categorization
Cybersecurity Dataset	Includes phishing, malware, and cybersecurity-related textual data for threat classification
Educational Dataset	Contains educational articles and e-learning content for academic text classification
Batch Text Dataset	Includes multiple text records processed together for batch classification testing
Uploaded File Dataset	Contains .txt and .csv files uploaded by users for large-scale text analysis
Preprocessed Text Dataset	Stores cleaned and tokenized text generated after NLP preprocessing operations

Example dataset record

text_id	text_content	source_name	source_type	extracted_keywords	entities	relevance_score	timestamp	status
1	AI improves healthcare systems.	TechNews	News	AI, Healthcare	AI	0.95	2026-05-25	Processed
2	Product quality is excellent.	Amazon	Review	Product Quality	Product	0.91	2026-05-25	Processed
3	Hackers launched phishing attacks.	Cyber Blog	Blog	Phishing Attack	Cybersecurity	0.93	2026-05-25	Classified
4	Football team won the match.	Sports Daily	News	Football Team	Sports	0.88	2026-05-25	Processed
5	You won a free mobile phone.	Spam Messages	Spam	Free Mobile	Spam	0.97	2026-05-25	Detected

8.6 Source Code

```
from django.shortcuts import render, redirect
from django.http import JsonResponse, HttpResponse
from django.views.decorators.csrf import csrf_exempt
from django.views.decorators.http import require_GET,
require_POST
from django.core.files.storage import default_storage
from django.conf import settings
from django.contrib import messages
import json
```

```

import os
import csv
import torch
import gc
import logging
import chardet
import uuid
import time

from datetime import datetime
from typing import List, Dict, Any
from pathlib import Path

from .forms import TextClassificationForm,
BatchClassificationForm, FileUploadForm

from .ml_models.bert_bgca import BERT_BGCA_Classifier
from .ml_models.openai_classifier import OpenAIClassifier
from .ml_models.model_utils import ModelEnsemble, clear_memory,
check_memory

# Set up logging
logger = logging.getLogger(__name__)

# Global model instances (lazy loading)
_bert_model = None
_openai_model = None
_ensemble = None

# Progress tracking store (use Redis/cache in production)
upload_progress_store = {}

# Model loading functions
def get_bert_model():
    """Lazy loading for BERT-BGCA model"""
    global _bert_model

```

```

if _bert_model is None:
    try:
        logger.info("Loading BERT-BGCA model...")

        # Check if pre-trained model exists
        model_path = os.path.join(settings.MODEL_CACHE_DIR,
'bert_bgca_model.pth')

        if os.path.exists(model_path):
            _bert_model =
BERT_BGCA_Classifier.load_model(model_path)
            logger.info("BERT-BGCA model loaded from
cache")
        else:
            _bert_model = BERT_BGCA_Classifier()
            logger.info("BERT-BGCA model initialized
(untrained)")

            # Log model info
            model_info = _bert_model.get_model_info()
            logger.info(f"Model info: {model_info}")

    except Exception as e:
        logger.error(f"Failed to load BERT model:
{str(e)}", exc_info=True)
        return None
    return _bert_model

```

```

def get_openai_model():
    """Lazy loading for OpenAI model"""
    global _openai_model

```

```

    if _openai_model is None and settings.OPENAI_API_KEY:
        try:
            logger.info("Loading OpenAI model...")
            _openai_model =
OpenAIClassifier(api_key=settings.OPENAI_API_KEY)
            logger.info("OpenAI model loaded successfully")
        except Exception as e:
            logger.error(f"Failed to load OpenAI model:
{str(e)}", exc_info=True)
            return None
    return _openai_model

def get_ensemble():
    """Lazy loading for ensemble model"""
    global _ensemble
    if _ensemble is None:
        _ensemble = ModelEnsemble()
        bert_model = get_bert_model()
        openai_model = get_openai_model()

        if bert_model:
            _ensemble.add_model(bert_model, weight=1.5)
            logger.info("Added BERT model to ensemble")
        if openai_model:
            _ensemble.add_model(openai_model, weight=1.0)
            logger.info("Added OpenAI model to ensemble")
    return _ensemble

```

```

#
# Progress tracking functions

def update_progress(progress_id: str, **kwargs):
    """Update progress for a specific upload"""
    if progress_id not in upload_progress_store:
        upload_progress_store[progress_id] = {
            'start_time': time.time(),
            'last_update': time.time()
        }

    upload_progress_store[progress_id].update(kwargs)
    upload_progress_store[progress_id]['last_update'] =
time.time()

    # Calculate percentage
    total = upload_progress_store[progress_id].get('total', 0)
    processed =
upload_progress_store[progress_id].get('processed', 0)

    if total > 0:
        percentage = (processed / total) * 100
        upload_progress_store[progress_id]['percentage'] =
round(percentage, 2)

    # Calculate ETA
    elapsed =
upload_progress_store[progress_id]['start_time'] -
time.time()

    if processed > 0:
        eta = (elapsed / processed) * (total - processed)

```

```
        upload_progress_store[progress_id]['eta_seconds']  
= round(eta)
```

```
# Clean up old progress entries (older than 1 hour)  
cleanup_old_progress()
```

```
def cleanup_old_progress():  
    """Remove progress entries older than 1 hour"""  
    current_time = time.time()  
    to_delete = []
```

CHAPTER – 9

PERFORMANCE ANALYSIS

Algorithms Used in the System

The project “Natural Language Processing Classification of Web Texts Combined with Deep Learning” uses several advanced algorithms and techniques to preprocess, analyze, and classify web-based textual data efficiently. These algorithms improve contextual understanding, semantic analysis, and classification accuracy within the system.

1.BERT (Bidirectional Encoder Representations from Transformers)

BERT is the main algorithm used for generating contextual word embedding and semantic feature extraction from textual data. It understands the meaning of words based on both left and right context within a sentence, which improves text understanding and classification performance. BERT helps the system generate high-quality feature representations from web texts.

2. BIGRU (Bidirectional Gated Recurrent Unit)

BIGRU is a Deep Learning algorithm used for learning sequential and contextual relationships in textual data. It processes text in both forward and backward directions to capture long-term dependencies and sentence meaning effectively. BIGRU improves contextual understanding and classification accuracy in web text analysis.

3. CNN (Convolutional Neural Network)

CNN is used for extracting important local features, keywords, and textual patterns from web texts. It helps identify meaningful phrases and hidden structures within the textual data. CNN improves feature extraction efficiency and supports accurate classification performance.

4. Attention Mechanism

The Attention Mechanism helps the model focus on the most important words and features present in the input text while ignoring irrelevant information. It improves semantic understanding, feature selection, and overall prediction accuracy during the classification process.

5. NLP Preprocessing Algorithms

Natural Language Processing preprocessing techniques such as tokenization, stop-word removal, stemming, and lemmatization are used to clean and prepare textual data before classification. These techniques improve text quality, reduce noise, and enhance model performance.

6. Softmax Classification Algorithm

The Softmax algorithm is used in the final output layer of the Deep Learning model to calculate probability scores for each classification category. It helps determine the most appropriate class label for the given web text input based on prediction probabilities.

7. Feature Extraction Techniques

Feature extraction algorithms are used to convert textual data into numerical vector representations that can be processed by Deep Learning models. Techniques such as BERT embedding and contextual feature generation help improve semantic analysis and classification efficiency.

Algorithm Comparison Table

Algorithm	Purpose	Advantages	Performance
BERT	Generates contextual word embedding and semantic features from text	Better contextual understanding and semantic analysis	High Accuracy
BIGRU	Learns sequential relationships in textual data	Captures forward and backward context efficiently	High Performance
CNN	Extracts important local textual features and keywords	Fast feature extraction and pattern detection	Efficient Classification
Attention Mechanism	Focuses on important words and relevant features	Improves feature selection and prediction accuracy	Improved Precision
Softmax Classifier	Predicts final text category probabilities	Simple and effective multi-class classification	Reliable Prediction

CHAPTER – 10

RESULT VALIDATION

The output analysis of the project focuses on evaluating the classification results generated by the NLP and Deep Learning system. After processing the input text using preprocessing techniques, BERT embedding, BIGRU, CNN, and Attention mechanisms, the system produces outputs such as predicted text category, confidence score, extracted keywords, and classification status. The output analysis helps determine how accurately the system classifies different types of web texts such as spam messages, sentiment data, news articles, cybersecurity content, educational text, and social media posts.

The generated outputs are analyzed using performance evaluation metrics such as accuracy, precision, recall, and F1-score to measure the effectiveness of the classification model. The confidence score indicates how strongly the system predicts a particular category, while the extracted keywords help understand important textual features identified during classification. The system also supports batch output analysis for multiple text records and displays results in a user-friendly format through the web interface. Overall, the output analysis shows that the integration of NLP and Deep Learning improves contextual understanding, classification accuracy, semantic analysis, and automated decision-making capabilities in web text classification systems.

10.1 Testing

Testing is performed to ensure that the system works accurately, efficiently, and reliably during web text classification. The testing process verifies whether all modules such as text input, file upload, NLP preprocessing, BERT embedding generation, Deep Learning classification, database storage, and result display are functioning correctly. Different types of textual inputs including single text, batch text, and uploaded files are tested to check the performance and stability of the system.

Functional testing is performed to verify that each module performs its intended operation correctly, while integration testing checks the communication between modules such as the NLP processor, BERT model, Deep Learning classifier, and database system. Performance testing is carried out to evaluate system speed, classification accuracy, and large file processing

capability. Accuracy testing measures how correctly the system predicts text categories using evaluation metrics such as accuracy, precision, recall, and F1-score. User interface testing ensures that the web application is user-friendly, responsive, and capable of displaying classification results clearly. Overall, testing helps improve system reliability, reduce errors, enhance classification performance, and ensure smooth execution of the NLP and Deep Learning-based web text classification system.

Types of Testing Performed

1. Unit Testing

Unit testing is used to test individual modules separately, such as the NLP preprocessing module, BERT embedding module, and Deep Learning classification module. It helps identify errors within specific components of the system.

2. Integration Testing

Integration testing verifies the communication and data flow between different modules such as the frontend, backend, NLP processor, database, and classification engine. It ensures that all modules work together properly.

3. Functional Testing

Functional testing is performed to verify whether all modules of the system work correctly according to the project requirements. It checks operations such as text input, batch classification, file upload, model selection, and result generation.

4. Security Testing

Security testing ensures that uploaded files, textual data, and system operations are protected from unauthorized access, malicious input, and data handling vulnerabilities.

5. Performance Testing

Performance testing evaluates system speed, response time, scalability, and efficiency while processing large volumes of web text data or uploaded files. It checks whether the system performs well under heavy workloads.

6. User interface Testing

User Interface testing checks whether the web application is user-friendly, responsive, and easy to navigate. It ensures proper display of text inputs, file uploads, model selection, and classification results.

7. Database Testing

Database testing verifies the storage, retrieval, and management of user inputs, uploaded files, preprocessing data, and classification results within the database system.

8. Security Testing

Security testing ensures that uploaded files, textual data, and system operations are protected from unauthorized access, malicious input, and data handling vulnerabilities

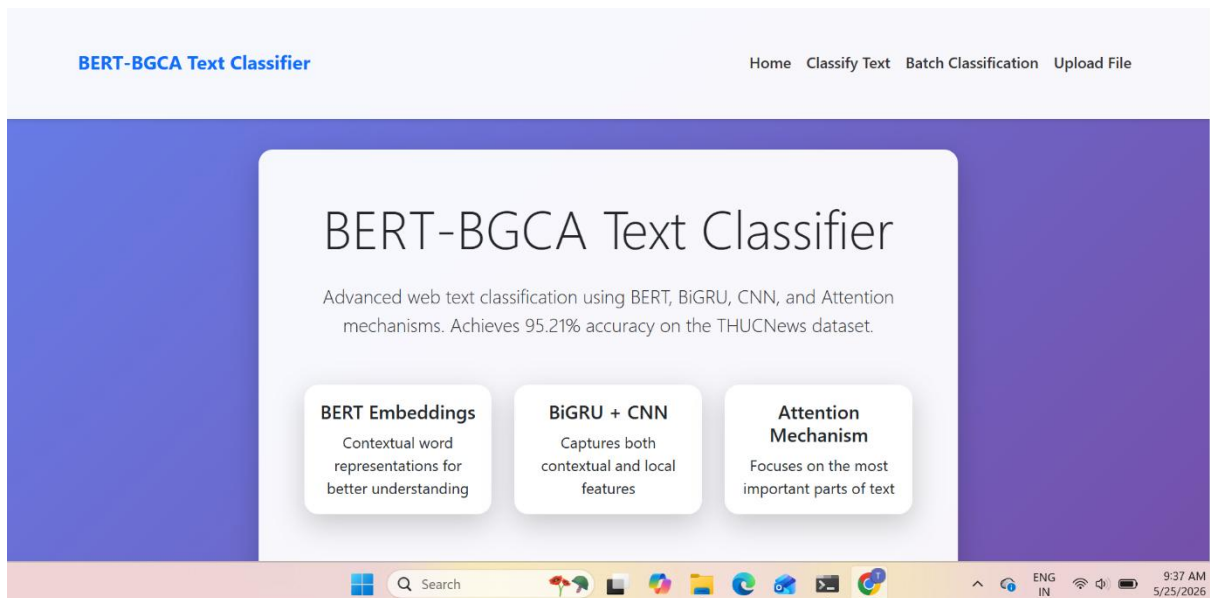
Test Cases

Test Case Type	Test Case ID	Function	Expected Results	Actual Results	Low Priority	High Priority
Functional Testing	TC_01	Enter single web text	Text should be accepted successfully	Text accepted		✓
Functional Testing	TC_02	Select BERT-BGCA model	Selected model should be applied	Model selected		✓
File Upload Testing	TC_03	Upload .txt file	File should upload successfully	File uploaded		✓
Batch Testing	TC_04	Enter multiple texts	System should process all texts	Batch text processed		✓
NLP Testing	TC_05	Preprocess input text	Text should be cleaned and tokenized	Text preprocessed		✓
Classification Testing	TC_06	Predict text category	Correct category	Category displayed		✓

			should be displayed			
Output Testing	TC_07	Display result	Predicted class and confidence score should appear	Result displayed		✓
Database Testing	TC_08	Save classification result	Result should be stored in database	Result saved	✓	
UI Testing	TC_09	Navigate between pages	Pages should open correctly	Navigation successful	✓	
Validation Testing	TC_10	Submit empty text box	System should show validation message	Validation shown		✓

10.2 Screenshots

Loading page- BERT-BGCA TEXT CLASSIFIER –About Page



Single Text Classification Page- classify text into single text

The screenshot shows a web browser window with the address bar displaying "127.0.0.1:8000/classify/". The page title is "BERT-BGCA Text Classifier". The navigation bar includes links for "Home", "Classify Text", "Batch Classification", and "Upload File". The main content area is titled "Classify Single Text" and contains a form with two sections: "Text to Classify" and "Classification Model". The "Text to Classify" section has a large text input field with the placeholder "Enter text to classify...". The "Classification Model" section has a dropdown menu currently showing "BERT-BGCA (Recommended)". Below the dropdown, there is explanatory text: "BERT-BGCA: Recommended for best accuracy", "OpenAI: Alternative using GPT", and "Ensemble: Combination of both". At the bottom of the form is a blue button labeled "Classify Text".

Batch wise classification text – Classifying text into multiple text

The screenshot shows the "Batch Text Classification" page. It features a large text input area at the top with the placeholder "Enter multiple texts (one per line)...". Below this input area, there is a smaller text input field with the placeholder "Enter one text per line". Underneath, there is a "Classification Model" dropdown menu set to "BERT-BGCA". At the bottom of the page is a blue button labeled "Classify All".

Large File Upload and Processing Page – large file uploading and converting into text

Upload Large File (Up to 150MB)

150MB
Maximum File Size

1M+
Lines Supported

Chunked
Processing

Upload File (up to 150MB)

Choose file No file chosen

Supported formats: .txt, .csv, .json, .tsv
Maximum size: 150MB
Each line will be truncated to 5000 characters

Classification Model
BERT-BGCA (Fast, GPU recommended)
BERT-BGCA is fastest for large files

Processing Chunk Size
Large (1000 lines) - Most
Larger = faster but more memory

Maximum Lines to Process
100,000 lines

Upload & Process File

Classified single text- I have classified one single text

Classify Single Text

Text to Classify

"The product quality is excellent and I am very satisfied with the service."

Classification Model
BERT-BGCA (Recommended)
BERT-BGCA: Recommended for best accuracy
OpenAI: Alternative using GPT
Ensemble: Combination of both

Classify Text

Classification Result

Text: "The product quality is excellent and I am very satisfied with the service."
Confidence Level:

After classifying – we get the classified result

Classification Result

Text: "The product quality is excellent and I am very satisfied with the service."
Confidence Level:

Category: sports
Category ID:

Confidence: 0.300
Method: rule_based_fallback

Batch text classification- taken multiple text

BERT-BGCA Text Classifier

HomeClassify TextBatch ClassificationUpload File

Batch Text Classification

Texts to Classify (one per line)

The product quality is excellent and I am very satisfied with the service.

The website performance is very poor and slow.

Congratulations! You won a free gift voucher. Click here now.

Artificial Intelligence is transforming modern industries.

Hackers launched a phishing attack on banking users.

The football team won the championship after a thrilling match.

Online learning platforms are improving education systems.

Scientists discovered a new technology for healthcare improvement.

Enter one text per line

Classification Model

Based on batch wise text- we get the result based on text what's the category ,confidence level and what method it is

Results				
#	Text	Category	Confidence	Method
1	The product quality is excellent and I am very satisfied with the service.	finance	0.350	bert_bgca
2	The website performance is very poor and slow.	finance	0.421	bert_bgca
3	Congratulations! You won a free gift voucher. Click here now.	finance	0.360	bert_bgca
4	Artificial Intelligence is transforming modern industries.	education	0.348	bert_bgca
5	Hackers launched a phishing attack on banking users.	finance	0.385	bert_bgca
6	The football team won the championship after a thrilling match.	finance	0.466	bert_bgca
7	Online learning platforms are improving education systems.	finance	0.522	bert_bgca
8	Scientists discovered a new technology for healthcare improvement.	finance	0.441	bert_bgca

CHAPTER – 11

FINAL SUMMARY

The project “Natural Language Processing Classification of Web Texts Combined with Deep Learning” is an intelligent web-based system developed for efficient classification and analysis of web textual data using advanced NLP and Deep Learning techniques. The system is designed to process different forms of textual input such as single text, batch text, and uploaded large files through a user-friendly web interface. Natural Language Processing techniques such as text cleaning, tokenization, stop-word removal, stemming, and lemmatization are used to preprocess and improve the quality of textual data before classification. The project integrates BERT-based contextual embedding along with Deep Learning models such as BiGRU, CNN, and Attention mechanisms to perform accurate semantic understanding and automated web text classification. The system supports model selection options such as BERT-BGCA, OpenAI, and Ensemble models for better prediction flexibility and performance. Classification results such as predicted category, confidence score, extracted keywords, and processed outputs are displayed clearly through the result management module and can also be saved or downloaded for future use. The project also includes database management, large file handling, batch processing, UML modeling, testing, and performance evaluation techniques to ensure scalability, reliability, and efficient execution. Overall, the project provides a smart, scalable, and highly accurate solution for automated web text classification and textual information analysis using modern NLP and Deep Learning technologies.

11.1 Project Outcome

The outcome is the development of an intelligent and efficient web-based text classification system capable of automatically analyzing and categorizing large volumes of web textual data with high accuracy. The project successfully integrates Natural Language Processing techniques with Deep Learning models such as BERT, BiGRU, CNN, and Attention mechanisms to improve contextual understanding, semantic analysis, and automated prediction capabilities. The system supports single text classification, batch text processing, and large file uploads, making it scalable and suitable for real-time web text analysis applications. It generates accurate outputs such as predicted text categories, confidence scores, extracted keywords, and processed classification results through an interactive user interface. The project

also improves processing speed, classification performance, and feature extraction efficiency while reducing manual effort in text analysis. Overall, the system provides a reliable, scalable, and intelligent solution for automated web text classification and information processing using modern NLP and Deep Learning technologies.

11.2 Future Enhancements

It focuses on improving classification accuracy, scalability, automation, and intelligent decision-making capabilities. In future developments, more advanced transformer-based models such as GPT, ROBERTA, XLNET and multilingual BERT can be integrated to improve semantic understanding and support multiple languages. The system can be enhanced to perform real-time web scraping and live text classification from social media platforms, news websites, blogs, and online forums automatically. Additional features such as sentiment analysis, fake news detection, emotion recognition, topic modeling, and recommendation systems can also be integrated to extend the functionality of the application. Cloud deployment and distributed processing techniques can be implemented to improve scalability and handle very large-scale datasets efficiently. Future versions may also include voice-to-text processing, chatbot integration, AI-based report generation, and advanced data visualization dashboards for better user interaction and analysis. Overall, these enhancements can make the system more intelligent, scalable, secure, and suitable for advanced real-world NLP and Deep Learning applications.

CHAPTER – 12

RESOURCE MATERIALS

1. Devlin, J., Chang, M. W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. Proceedings of NAACL-HLT, 4171–4186.
2. Vaswani, A., Shazeer, N., Parmar, N., et al. (2017). Attention Is All You Need. Advances in Neural Information Processing Systems (NeurIPS), 5998–6008.
3. Kim, Y. (2014). Convolutional Neural Networks for Sentence Classification. Proceedings of EMNLP, 1746–1751.
4. Cho, K., Van Merriënboer, B., Gulcehre, C., et al. (2014). Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation. Proceedings of EMNLP, 1724–1734.
5. Hochreiter, S., & Schmidhuber, J. (1997). Long Short-Term Memory. Neural Computation, 9(8), 1735–1780.
6. Mikolov, T., Chen, K., Corrado, G., & Dean, J. (2013). Efficient Estimation of Word Representations in Vector Space. *Proceedings of ICLR*.
7. Pennington, J., Socher, R., & Manning, C. D. (2014). GloVe: Global Vectors for Word Representation. Proceedings of EMNLP, 1532–1543.
8. Yang, Z., Yang, D., Dyer, C., et al. (2016). Hierarchical Attention Networks for Document Classification. Proceedings of NAACL-HLT, 1480–1489.
9. Liu, Y., Ott, M., Goyal, N., et al. (2019). RoBERTa: A Robustly Optimized BERT Pretraining Approach. arXiv preprint arXiv:1907.11692.
10. Brown, T., Mann, B., Ryder, N., et al. (2020). *Language Models are Few-Shot Learners*. Advances in Neural Information Processing Systems (NeurIPS), 1877–1901.